

WinDbg / cdb Cheat Sheet — x86 Exploit Dev (OSED)

Default number base is **hex**. Write decimal as 0n100. Registers are **@eax**, pseudo-regs **\$teb**. **poi(x)** = deref pointer at x. Comment in scripts with **\$\$** or *****.

Launch / Attach / Symbols

Command	Purpose
<code>cdb -o -G target.exe args</code>	Launch + debug children, ignore final break. (~g skips the <i>initial</i> break — usually leave it off.)
<code>cdb -cf script.wds target.exe</code>	Run script at initial loader break, then continue.
<code>windbg -p <pid> / -pn name.exe</code>	Attach to running process by pid / name.
<code>.sympath srv*c:\sym*https://msdl.microsoft.com/download/symbols</code>	Set Microsoft symbol path.
<code>.reload /f</code>	Force-reload symbols for all modules.
<code>lm / lmf / lm m vuln*</code>	List modules / with paths / filtered.
<code>x ntdll!*Ldr*</code>	Search symbols by wildcard.
<code>ln 0x76f7db6b</code>	Nearest symbol to an address.

Execution Control

Command	Purpose
<code>g</code>	Go (run). <code>g 0x401000</code> = run to address.
<code>p / t</code>	Step over / step into (one instruction).
<code>pa 0x401050</code>	Step (over) until address is reached.
<code>pt / pc</code>	Step to next <code>ret</code> / next <code>call</code> .
<code>gu</code>	Go up — run until current function returns.
<code>.restart</code>	Restart the target from scratch.
<code>q / qd</code>	Quit (kill) / quit and detach.

Breakpoints

Command	Purpose
<code>bp module+0x1821</code>	Software BP at module RVA (module already loaded).
<code>bp kernel32!WinExec</code>	BP on a symbol.
<code>bu module!func</code>	Deferred BP — resolves when module loads.
<code>ba w4 0x402000</code>	Hardware BP: break on 4-byte w rite (also <code>r/e/i</code>).
<code>bp x "command; g"</code>	BP that runs a command list then continues.
<code>bp x ".if (poi(@esp+8)==0x41) {} .else {gc}"</code>	Conditional BP (break only when condition holds).
<code>bl / bc * / bd 0 / be 0</code>	List / clear all / disable / enable BP.
<code>bp x 1000</code>	Pass count — break only on the 1000th hit.

Registers and Flags

Command	Purpose
<code>r</code>	Dump all registers + flags + next instruction.
<code>r eax / r eax=0x41414141</code>	Show / set a single register.
<code>r \$t0=0x100</code>	Set a scratch pseudo-register (<code>\$t0..\$t19</code>).
<code>flags</code>	<code>zr=ZF cy=CF ov=OF pl/mi=sign</code> — shown in <code>r</code> .

Examining Memory

Command	Purpose
<code>db esp L20</code>	Bytes + ASCII, 0x20 of them.
<code>dd esp L40</code>	DWORDs (the stack-reading workhorse).
<code>dps esp L20</code>	DWORDs with symbol resolution (find return addr).
<code>dc, dw, dq</code>	DWORD+ASCII, WORDs, QWORDs.
<code>da / du addr</code>	ASCII / Unicode string at address.
<code>dt ntdll!_PEB @\$peb</code>	Dump a typed structure (if symbols present).
<code>!address esp</code>	Region info + permissions for an address.
<code>eb addr 90 90 / ed addr 0x..</code>	Edit bytes / dword in memory.

Disassembly

Command	Purpose
<code>u eip / u addr L20</code>	Disassemble forward (default 8 instrs).
<code>ub addr</code>	Disassemble backward (find the call before a <code>ret</code> addr).
<code>uf module!func</code>	Disassemble a whole function.
<code>u poi(@esp)</code>	Disassemble at the address on top of stack.

Stack and Call Frames

Command	Purpose
k / kb / kp .frame N dv	Call stack / with first 3 args / with full params. Switch to frame N (then dv, dd ebp are in its context). Local variables (needs private symbols — fails on stripped).

Stripped binary (no dv): read the prologue. sub esp,0x208 = local size; [ebp-N] = locals, [ebp+8..] = args, [ebp+4] = saved return address.

Searching Memory

Command	Purpose
s -a 0 L780000000 "TRUN" s -u 0 L780000000 "calc" s -b 0 L780000000 90 90 90 90 s -d 0 L780000000 0x41414141	Search all memory for ASCII string. Search for Unicode string. Search for a byte pattern (e.g. NOP sled). Search for a DWORD value.

SEH / Exceptions (OSED-critical)

Command	Purpose
!exchain sxe av / sxd av sxe ld:module g (after AV) .exr -1 / .ecxr	Show the SEH chain (handler addresses). Break on / ignore access violations. Break when a specific module loads. First chance vs second chance — pass once to reach handler. Show last exception record / its context.

Exploit-Dev Recipes

```
Pattern -> offset (no Mona):
  send cyclic pattern, crash, then:   r eip
  msf-pattern_offset -q <eip value>   (or !mona findmsp)

Find a jmp esp / call esp return address:
  s -b 0x62500000 L?1000 ff e4      $$ jmp esp opcode
  s -b 0x62500000 L?1000 ff d4      $$ call esp
  !mona jmp -r esp -cpb '\x00\x0a\x0d'  $$ with bad-char filter

POP/POP/RET for SEH:
  !mona seh -cpb '\x00\x0a\x0d'

Dump a buffer to disk (bad-char capture):
  !writemem C:\dbg\dump.bin poi(@esp+4)+0x7d6 (poi(@esp+4)+0x7d6)+0x200

Compare bytes in memory vs a clean array:
  !mona compare -f C:\mona\bytearray.bin -a <addr>

Verify a candidate dump (Python side):  dump[:4] == b"\xbc\x0f\xbc\x0f"
```

Pseudo-Registers and Operators

Token	Meaning
@eax @esp @eip	Register values (the @ avoids symbol lookup).
\$teb \$peb	TEB / PEB base for current thread/process.
\$ra \$ip \$csp	Return address / instruction ptr / stack ptr.
poi(x)	Pointer dereference: the DWORD stored at x.
by(x) wo(x) dwo(x)	Byte / word / dword at x.
? expr	Evaluate an expression. ? poi(@esp+4)+0n2006.
0n100 / 0x64	Decimal 100 / hex 100. Default base is hex .

Logging and Scripting

Command	Purpose
.logopen c:\dbg\log.txt / .logclose .printf "eip=%x\n", @eip .echo BADCHAR_CRASH \$\$<c:\s.wds / \$><c:\s.wds .foreach (a {!some}) { u a } .shell -ci "cmd" cmd /c move ...	Capture session output to a file. Formatted output in a command/script. Emit a marker string (for harness detection). Run a script file (\$>< allows block commands). Iterate over tokens of another command's output. Run a host shell command (e.g. atomic dump rename).